装 订 线

专业_____ 班级_____ 姓名_____ 学号_____ 序号_____

广 东 财 经 大 学 答 题 纸（格式二）

课程 数据结构课程设计 2025－2026 学年第 2 学期

成绩_____ 评阅人_____

评分标准：

课程设计文档不清晰，无法阅读，得 0 分。

课程设计文档务必填上学号与姓名。

检查项目	检查要点	权重	得分
答辩陈述	答辩陈述与逻辑表达，主题明确、语言流畅、陈述重点、回答逻辑等		
答辩提问	内容熟悉度与问题回答，对程序/报告内容的熟悉程度、反应敏捷、准确回答问题等		
程序部分	程序实现与代码质量，完成要求功能、输入输出正确、注释充分、额外功能等		
测试部分	测试用例与任务完成度，完成基本任务测试、增加额外测试用例或条件等		
报告内容撰写	文档规范与报告质量，格式规范、内容齐全、论述充分、图文并茂、行文流畅等		
合计			

《最小生成树的应用：室内布线》项目报告

目　　录

一、设计要求

1.1 问题描述

1.2 需求分析

1.2.1 功能需求

1.2.2 性能需求

二、概要设计

2.1 主界面设计

2.2 存储结构设计

2.3 系统功能设计

三、模块设计

3.1 系统模块设计

3.2 系统子程序及功能设计

3.3 函数主要调用关系图

四、详细设计

4.1 数据类型定义

4.2 系统主要子程序详细

五、测试分析

5.1 常规无障碍测试

5.2 直接阻挡绕行测试

5.3 完全隔离测试

5.4 进阶功能测试（图形化界面与文件读取）

六、用户手册

6.1 环境与运行准备

6.2 基础版本交互说明**(indoor_wiring.py)**

6.3 进阶图形化界面说明**(app_gui.py)**

附件（源程序清单）

一、设计要求

1.1 问题描述

本课程设计旨在解决室内装修中的电线布线优化问题。在实际装修中，需要在多个固定的插座位置之间铺设电线，且受限于承重墙、门窗等实体障碍物，电线铺设需满足特定工程规范规则。

具体问题抽象为：给定二维平面上的 n 个插座坐标点（需连通的节点）和 m 个门（不可穿透的矩形障碍物），要求计算在满足“电线只能水平或垂直铺设（曼哈顿距离）且不得直接穿过门区域”的约束下，将所有插座连通所需的最短电线总长度，并输出具体布线方案。该问题本质上是带障碍物约束的完全图最小生成树(MST)问题。

1.2 需求分析

1.2.1 功能需求

（1）数据输入：准许用户输入插座的(x,y)坐标集合与矩形障碍物（门）的对角线坐标。

（2）合法路径生成：能够判断两点间的水平与垂直走线是否与障碍物相交，若相交则自动计算从门角绕行的合法最短路径。

（3）最小生成树计算：基于各个节点之间的合法最短路径，计算连接所有节点的最小总长。

（4）结果可视化呈现：输出最短电线总长度，并通过图形化界面直观展示插座位置、障碍物以及布线轨迹。

1.2.2 性能需求

（1）时间复杂度要求：插座数量通常≤30 时，程序响应时间需在亚秒级，核心算法时间复杂度需控制在 $O(n^2 \log n)$级别。

（2）空间复杂度要求：程序应合理开辟内容，空间复杂度不超过 $O(n^2)$。

二、概要设计

2.1 主界面设计

系统包括两种交互模式。基础版采用控制台(CLI)交互，提示用户依次输入插座数量与坐标及门的位置；进阶版采用图形用户界面(GUI)，提供直观的数据填写表单与一张基于 Matplotlib 生成的二维坐标画布，用于实时展示布线结果图。

2.2 存储结构设计

(1) **节点(Point)与障碍物(Rect)：**采用自定义类进行封装，分别记录二维坐标属性。

(2) **图的边集：**采用列表(List)存储所有合法的边，列表元素为四元组（起点编号，终点编号，权值边长，路径实例）。

(3) **并查集(DSU)：**采用一维数组(List)实现，parent 数组存储当前节点的父节点，rank 数组用于按秩合并以优化树深度。

2.3 系统功能设计

系统主要分为四大功能模块：

(1) **数据解析模块：**负责读取与校验用户输入的坐标。

(2) **路径权值计算模块：**核心几何计算，判断线段与矩形相交状态，并生成直接或绕行的曼哈顿路径。

(3) **图论求解模块：**使用 Kruskal 算法求完全图（剔除死路边）的最小生成树。

(4) **可视化展示模块：**渲染点、面、线的二维图形。

三、模块设计

3.1 系统模块设计

系统被划分为四个层次：

输入层→几何计算层→连通计算层→输出展示层

3.2 系统子程序及功能设计

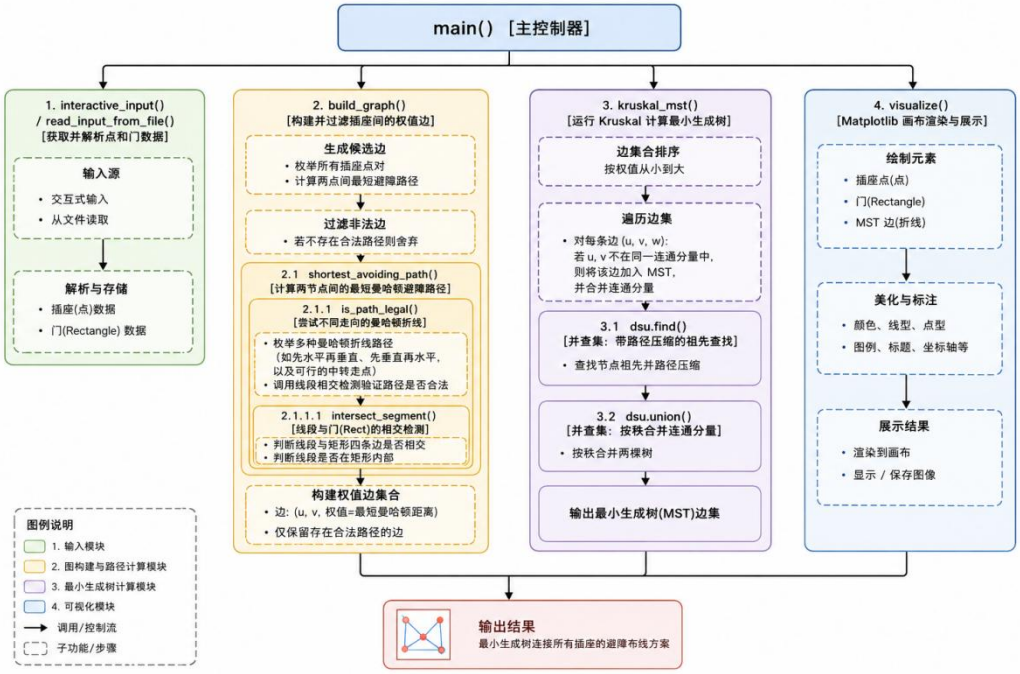
(1) **intersect_segment()：**判断给定线段是否穿透门矩形。

(2) **shortest_avoiding_path()：**尝试直接路径或四角绕行路径，寻找两插座间不跨越障碍物的最短曼哈顿路径。

(3) **build_graph()：**遍历所有插座组合，调用路径查找模块，建立带权无向图的边集。

(4) **kruskal_mst()：**基于边集升序排序和并查集，查找并返回最小生成树。

3.3 函数主要调用关系图



图一-函数主要调用关系图

四、详细设计

4.1 数据类型定义

(1) 插座点类型（Point 类）

该类用于表示二维空间中一个插座或拐点的坐标。

x（float 类型）：表示节点的水平 x 坐标。

y（float 类型）：表示节点的垂直 y 坐标。

方法__eq__：重载了等于号判定。考虑到浮点数运算可能产生的精度丢失（如 0.1+0.2 != 0.3），采用绝对差值< 1e-9 作为两点重合的判断标准，保障核心几何算子的精确性。

(2) 障碍物类型（Rect 类）

该类用于表示平面内一个“门”所占据的矩形物理隔离区域。为了防止用户输入坐标时左下与右上坐标颠倒，本类在初始化时即做自动校正。

x1,y2（float 类型）：严格存入该门的最小 x 坐标与最小 y 坐标（即左下角基准点）。

x2,y2（float 类型）：严格存入该门的最大 x 坐标与最大 y 坐标（即右上角极限点）。

方法 contains_point(p)：校验某个坐标点是否严格位于矩形障碍物内部。

方法 intersect_segment(p1,p2)：核心边界相交检测算法。用于判断参数线段（从 p1 到 p2）是否穿过门区域的四条边界中的任意一条。

(3) 并查集结构（DSU 类）

该类主要用来支持 Kruskal 算法高效检测环路（圈）。

parent（List[int]数组）：记录当前结点的“父节点”，初始化时每个结点的父节点是其身。

rank（List[int]数组）：记录当前树的高度（秩），作为优化项，在合并时将较浅的树合并到较深的树下，保持树的扁平化。

(4) 图的边结构（Tuple 形式）

采用无向加权边的方式存储两个插座连通的可能性。不单独建类，而是使用四元组（Tuple）存储于 edges 列表中：

格式为：(u, v, weight, path_points)

u,v（int 类型）：连线的起点与终点插座在数组中的下标索引。

weight（float 类型）：该条边连通所需的真实最短电线走向长度。

path_points（List[Point]）：电线折点序列。记录连线的具体走向轨迹，供末端的可视化渲染模块循迹画图。

4.2 系统主要子程序详细

(1) 基于曼哈顿距离的避障寻路详细设计：

两点(x_1,y_1)与(x_2,y_2)间存在两种直接的曼哈顿折线：先转水平后转垂直，或先转垂直后水平。子程序依次检查这两条路径是否触碰所有定义的 Rect。若全受阻，则收集所有的门的四个角点，让线段尝试分两步：起点→角点，角点→终点，从中选取出最小的权值为该条边的合法权值。

(2) Kruskal 最小权值连通详细设计：

通过 build_graph 得到的所有合法连线后，依长度进行升序排序。遍历边，若两端点所在并查集不连通，则合并集合并加入选中边集中；当选中边数 n-1 时，算法提前终止，完成 MST 构建。

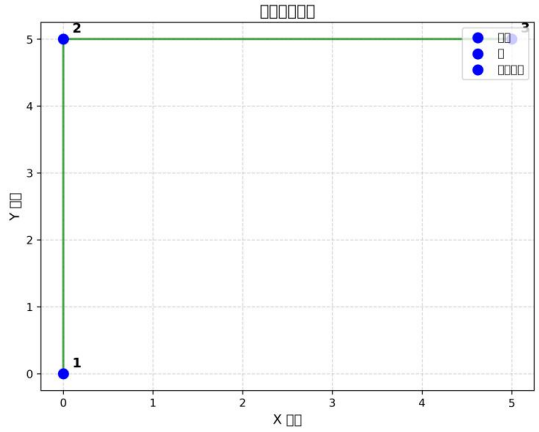
五、测试分析

5.1 常规无障碍测试

输入：插座(0,0)，(0,5)，(5,5)，无门。

预期输出：形成长度为 10 的 L 型连线。

输出结果：



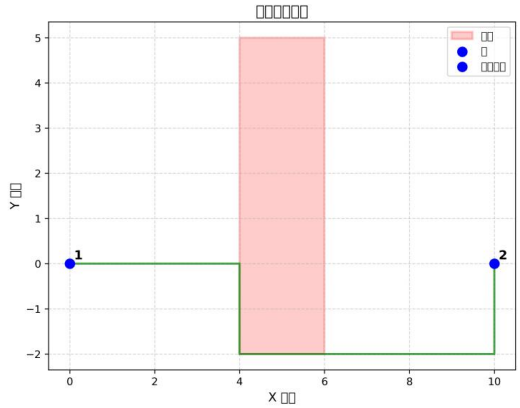
图二-测试 1 可视化输出结果

5.2 直接阻挡绕行测试

输入：插座 A(0,0)，插座 B(10,0)。在坐标(4,-2)到(6,5)增加一扇门。

预期输出：电线无法直接穿过 x=4 到 x=6 的区域，应当向上或向下绕过门的边缘。

输出结果：



图三-测试 2 可视化输出结果

5.3 完全隔离测试

输入：插座被门构成的矩形完全包裹。

预期输出：提示图不连通，无法布线。

输出结果：

```
正在计算插座间的最短可行路径……
进度: 1/1 (100.0%)
路径计算完成!
错误：没有任何两个插座可以连通，请检查门是否阻断了所有路径。

进程已结束，退出代码为 0
```

图四-测试 3 完全隔离触发异常输出结果

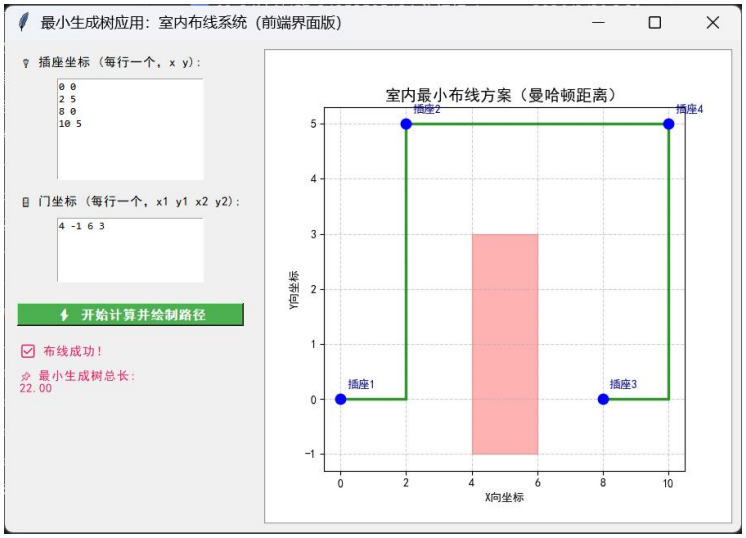
5.4 进阶功能测试（图形化界面与文件读取）

测试目的：验证系统进阶版本(app_gui.py)的前端交互能力，以及大规模数据下的文件解析能力。

输入：启动 GUI 程序，并在左侧面板坐标输入区域填写测试用例，按“开始计算并绘制路径”按钮。

预期输出：右侧空白画板实时渲染出基于 matplotlib 封装的 TKinter 画布，包含障碍物半透明红色遮罩、插座节点的散点标记，以及连通各节点的绿色最小生成树曼哈顿连线。

输出结果：



图五-测试 4 系统 GUI 图形用户前端运行结果

六、用户手册

6.1 环境与运行准备

系统采用 Python 3 编写，需依赖第三方库进行数学计算与渲染。

(1) **前置安装：**请在终端中执行命令 pip install matplotlib 安装必要绘图库。

(2) **文件构成：**项目包含基础控制台版本 indoor_wiring.py 和进阶图形化版本

app_gui.py，用户可按需运行。

6.2 基础版本交互说明(indoor_wiring.py)

在开发环境中运行该脚本后，系统提供两种交互模式：

(1) **方式 A-命令台交互式输入：**按控制台文字提示，手动输入插座数量、逐行录入坐标，再输入门数量及坐标。程序计算完成后将文字输出路径明细，并弹出最终路线图。

(2) **方式 B-外部测试文件导入：**面向大规模或复杂测试场景下，用户可预先在同级目录下建立标准文本文件（如 data.txt）避免重复输入。

外部文件标准格式要求如下：

第 1 行：一个整数 N，表示插座的数量。

第 2 到 N+1 行：每行包含两个数字（空格隔开），代表单个插座的 X 和 Y 坐标。

第 N+2 行：一个整数 M，表示障碍物（门）的数量。

最后 M 行：每行包含四个数字（空格隔开），代表矩形对角的(x1,y1)和(x2,y2)。

6.3 进阶图形化界面说明(app_gui.py)

该版本封装了 Tkinter 视窗，极大优化了非技术人员的用户操作体验。

(1) **启动程序：**直接运行 app_gui.py 即可弹出操作视窗。

(2) **数据录入：**在主界面左侧的“插座坐标区”与“门坐标区”内，每行键录一个对象的坐标参数即可（系统已预设了默认演示数据）。

(3) **一键制图：**点击绿色的“开始计算并绘制路径”按钮。系统将自动在左下角展示最终的电线总长，并在右侧高清画板区动态渲染出直观的设计图纸。若遭遇“死路”无法连通的情况，系统将自动弹出警告弹窗提示用户方案不可行。

附件（源程序清单）

indoor_wiring.py:

```
import math
```

```
import sys


try:
    import matplotlib.pyplot as plt
    import matplotlib.patches as patches
    MATPLOTLIB_AVAILABLE = True
except ImportError:
    MATPLOTLIB_AVAILABLE = False



class Point:
    # 平面坐标
    def __init__(self, x, y):
        self.x = x
        self.y = y


    def __repr__(self):
        return f"({self.x:.2f}, {self.y:.2f})"


    def __eq__(self, other):
        return abs(self.x - other.x) < 1e-9 and abs(self.y - other.y) < 1e-9



class Rect:
    # 矩形（门）
    def __init__(self, x1, y1, x2, y2):
        self.x1 = min(x1,x2)
        self.y1 = min(y1,y2)
```

```
        self.x2 = max(x1,x2)
        self.y2 = max(y1,y2)

    def contains_point(self, p):
        return self.x1 < p.x < self.x2 and self.y1 < p.y < self.y2

    def intersect_segment(self, p1, p2):
        """
        判断 p1-p2 是否与矩形相交（包括边界）
        """
        # 如果线段两个端点有一个在矩形内，则必相交
        if self.contains_point(p1) or self.contains_point(p2):
            return True

        dx = p2.x - p1.x
        dy = p2.y - p1.y

        # 检查门与四条边的交点
        # 左边界
        if abs(dx) > 1e-9:
            t_left = (self.x1 - p1.x) / dx
            if 0 < t_left < 1:
                y = p1.y +  t_left * dy
                if self.y1 < y < self.y2:
                    return True
        # 右边界
        if abs(dx) > 1e-9:
            t_right = (self.x2 - p1.x) / dx
```

```
            if 0 < t_right < 1:
                y = p1.y + t_right * dy
                if self.y1 < y < self.y2:
                    return True
        # 下边界
        if abs(dy) > 1e-9:
            t_bottom = (self.y1 - p1.y) / dy
            if 0 < t_bottom < 1:
                x = p1.x + t_bottom * dx
                if self.x1 < x < self.x2:
                    return True
        # 上边界
        if abs(dy) > 1e-9:
            t_top = (self.y2 - p1.y) / dy
            if 0 < t_top < 1:
                x = p1.x + t_top * dx
                if self.x1 < x < self.x2:
                    return True

        return False

    @staticmethod
    def _segments_intersect(p1, p2, p3, p4):
        """判断两条线段是否严格相交（交点不是端点）"""
        def cross(o, a, b):
            return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x)

        c1 = cross(p1, p2, p3)
```

```
        c2 = cross(p1, p2, p4)
        c3 = cross(p3, p4, p1)
        c4 = cross(p3, p4, p2)

        return (c1 * c2 < 0) and (c3 * c4 < 0)


def manhattan_distance(p1, p2):
    # 曼哈顿距离
    return abs(p1.x - p2.x) + abs(p1.y - p2.y)


def is_path_legal(p1, p2, doors):
    """
    检查两种曼哈顿路径是否至少有一条不与门相交
    1：先水平后垂直（p1 -> （p2.x, p1.y） -> p2）
    2：先垂直后水平（p1 -> （p1.x, p2.y） -> p2）
    返回
    """
    # 路径 1
    mid1 = Point(p2.x, p1.y)
    legal1 = True
    for door in doors:
        if door.intersect_segment(p1, mid1) or
door.intersect_segment(mid1, p2):
            legal1 = False
            break
    len1 = manhattan_distance(p1, p2) if legal1 else float('inf')
```

```
    # 路径 2
    mid2 = Point(p1.x, p2.y)
    legal2 = True
    for door in doors:
        if door.intersect_segment(p1, mid2) or
door.intersect_segment(mid2, p2):
            legal2 = False
            break
    len2 = manhattan_distance(p1, p2) if legal2 else float('inf')

    if legal1 or legal2:
        if len1 <= len2:
            return True, len1, [p1, mid1, p2]
        else:
            return True, len2, [p1, mid2, p2]
    else:
        return False, float('inf'), []



def shortest_avoiding_path(p1, p2, doors):
    # 直接路径
    legal, dist, path_points = is_path_legal(p1, p2, doors)
    if legal:
        return dist, 'direct', path_points

    # 绕行，尝试门的四个角点
    detour_points = []
```

```
    for door in doors:
        corners = [
            Point(door.x1, door.y1),
            Point(door.x1, door.y2),
            Point(door.x2, door.y1),
            Point(door.x2, door.y2),
        ]
        for corner in corners:
            if not any(corner == p for p in detour_points):
                detour_points.append(corner)

    best_dist = float('inf')
    best_path = []
    for corner in detour_points:
        leg1_legal, leg1_dist, leg1_path = is_path_legal(p1, corner, doors)
        leg2_legal, leg2_dist, leg2_path = is_path_legal(corner, p2, doors)

        if leg1_legal and leg2_legal:
            total = leg1_dist + leg2_dist
            if total < best_dist:
                best_dist = total
                best_path = leg1_path[:-1] + leg2_path

    if best_dist != float('inf'):
        return best_dist, 'detour', best_path
    else:
        return float('inf'), 'none', []
```

```
def build_graph(points, doors, progress_callback=None):
    # 构建完全图的边列表，每条边为（u, v, weight）
    n = len(points)
    edges = []
    total_pairs = n * (n - 1) // 2
    processed = 0

    for i in range(n):
        for j in range(i+1, n):
            dist, _, path_points = shortest_avoiding_path(points[i],
points[j], doors)
            if dist != float('inf'):
                edges.append((i, j, dist, path_points))

            processed += 1
            if progress_callback and processed % max(1, total_pairs//20) ==
0:
                progress_callback(processed, total_pairs)
    return edges



# ---------- 并查集 ----------
class DSU:
    def __init__(self, size):
        self.parent = list(range(size))
        self.rank = [0] * size
```

```
    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, x, y):
        x_root = self.find(x)
        y_root = self.find(y)
        if x_root == y_root:
            return False

        if self.rank[x_root] < self.rank[y_root]:
            self.parent[x_root] = y_root
        elif self.rank[x_root] > self.rank[y_root]:
            self.parent[y_root] = x_root
        else:
            self.parent[y_root] = x_root
            self.rank[x_root] += 1
        return True


def kruskal_mst(num_nodes, edges):
    edges_sorted = sorted(edges, key=lambda e: e[2])
    dsu = DSU(num_nodes)
    mst_edges = []
    total_weight = 0.0
```

```
    for u, v, w, path in edges_sorted:
        if dsu.union(u, v):
            mst_edges.append((u, v, w, path))
            total_weight += w
            if len(mst_edges) == num_nodes - 1:
                break

    if len(mst_edges) != num_nodes - 1:
        return None, [] # 不连通
    return total_weight, mst_edges



def visualize(points, doors, mst_edges, title="室内布线方案"):
    # 可视化
    if not MATPLOTLIB_AVAILABLE:
        print("matplotlib 未安装，无法显示图形。")
        return

    fig, ax = plt.subplots(figsize=(8,6))

    # 画出门（矩形）
    for door in doors:
        rect = patches.Rectangle((door.x1, door.y1), door.x2 - door.x1,
door.y2 - door.y1, linewidth=2, edgecolor='red', facecolor='red',
alpha=0.2)
        ax.add_patch(rect)

    # 画出插座点(蓝色＋符号)
```

```
    for i, p in enumerate(points):
        ax.scatter(p.x, p.y, c='blue', s=80, zorder=5, label='插座')
        ax.text(p.x + 0.1, p.y + 0.1, str(i+1), fontsize=12,
fontweight='bold')

    # 绘制 MST 边(画出实际的曼哈顿折线)
    for u, v, w, path in mst_edges:
        xs = [p.x for p in path]
        ys = [p.y for p in path]
        ax.plot(xs, ys, 'g-', linewidth=2, alpha=0.7, label='布线路径')

    ax.set_title(title, fontsize=14)
    ax.set_xlabel("X 坐标", fontsize=12)
    ax.set_ylabel("Y 坐标", fontsize=12)
    ax.grid(True, linestyle='--', alpha=0.5)
    ax.legend(["插座", "门", "布线路径"], loc='upper right')
    plt.savefig("布线方案.png", dpi=300, bbox_inches='tight')
    print("图形已保存为：布线方案.png")
    plt.show()


def read_input_from_file(filename):
    with open(filename, 'r') as f:
        lines = [line.strip() for line in f if line.strip()]

    idx = 0
    n = int(lines[idx]); idx += 1
    points = []
```

```
    for _ in range(n):
        x, y = map(float, lines[idx].split())
        points.append(Point(x, y))
        idx += 1

    m = int(lines[idx]); idx += 1
    doors = []
    for _ in range(m):
        x1, y1, x2, y2 = map(float, lines[idx].split())
        doors.append(Rect(x1, y1, x2, y2))
        idx += 1

    return points, doors



def interactive_input():
    print("===== 室内布线系统（最小生成树，支持多个门） =====")
    n = int(input("请输入插座数量："))
    points = []
    for i in range(n):
        while True:
            try:
                x, y = map(float, input(f"插座{i+1}坐标(x y)：").split())
                points.append(Point(x, y))
                break
            except:
                print("输入错误，请重新输入")
```

```
    m = int(input("请输入门的数量："))
    doors = []
    for i in range(m):
        while True:
            try:
                x1, y1, x2, y2 = map(float, input("左下角(x1 y1) 和 右上角
(x2 y2)，空格分隔：").split())
                doors.append(Rect(x1, y1, x2, y2))
                break
            except:
                print("输入错误，请重新输入")

    return points, doors



def main():
    mode = input("选择输入方式：1-交互式输入  2-从文件读取（文件格式见说明）：
").strip()
    if mode == '2':
        filename = input("请输入文件名：").strip()
        try:
            points, doors = read_input_from_file(filename)
            print(f"成功读取 {len(points)} 个插座和 {len(doors)} 个门")
        except Exception as e:
            print(f"读取文件失败：{e}")
            return
    else:
        points, doors = interactive_input()
```

```
    if len(points) < 2:
        print("至少需要 2 个插座才能布线。")
        return

    print("\n 正在计算插座间的最短可行路径……")

    def progress(current, total):
        print(f"\r 进度: {current}/{total} ({current/total*100:.1f}%)",
end='', flush=True)

    edges = build_graph(points, doors, progress_callback=progress)
    print("\n 路径计算完成！")

    if not edges:
        print("错误：没有任何两个插座可以连通，请检查门是否阻断了所有路径。")
        return

    total_len, mst_edges = kruskal_mst(len(points), edges)
    if total_len is None:
        print("错误：所有插座无法通过可行路径连通（图不连通）。")
        return

    print(f"\n 最短电线总长度：{total_len:.2f}")
    print("\n 布线方案（插座编号从一开始，边及长度）：")
    for u, v, w, path in mst_edges:
        path_str = "->".join([f"({p.x:.1f}, {p.y:.1f})" for p in path])
        print(f" 插座{u + 1} 到插座 {v + 1} | 长度：{w:.2f} | 路径：
```

```
{path_str}")


    # 询问
    if MATPLOTLIB_AVAILABLE:
        show_graph = input("\n 是否显示图形化布线图？（y/n）:
").strip().lower()
        if show_graph == "y":
            visualize(points, doors, mst_edges)
    else:
        print("\n 提示：未安装 matplotlib，无法显示图形。")
        print("如需图形化展示，请在当前 Python 环境执行：pip install
matplotlib")



if __name__ == "__main__":
     main()


app_gui.py:


import os
import sys


# 【兼容补丁】动态获取当前运行该代码的 Python 解释器路径
current_env_path = sys.exec_prefix


# 组装寻找 tcl 和 tk 库的路径
tcl_path = os.path.join(current_env_path, 'Library', 'lib', 'tcl8.6')
tk_path = os.path.join(current_env_path, 'Library', 'lib', 'tk8.6')
```

```
# 智能判断：如果当前电脑存在这个文件夹（说明是带中文的 conda 环境），就修补路径
# 如果不存在，就什么都不干，让程序按正常的流程走
if os.path.exists(tcl_path) and os.path.exists(tk_path):
    os.environ['TCL_LIBRARY'] = tcl_path
    os.environ['TK_LIBRARY'] = tk_path

import tkinter as tk
from tkinter import messagebox
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import matplotlib.patches as patches



# ==================== 第 1 部分：核心算法区域 ====================
class Point:
    def __init__(self, x, y):
        self.x = x;
        self.y = y

    def __eq__(self, other):
        return abs(self.x - other.x) < 1e-9 and abs(self.y - other.y) < 1e-9



class Rect:
    def __init__(self, x1, y1, x2, y2):
        self.x1 = min(x1, x2);
        self.y1 = min(y1, y2)
```

```
        self.x2 = max(x1, x2);
        self.y2 = max(y1, y2)

    def contains_point(self, p):
        return self.x1 < p.x < self.x2 and self.y1 < p.y < self.y2

    def intersect_segment(self, p1, p2):
        if self.contains_point(p1) or self.contains_point(p2): return True
        dx = p2.x - p1.x;
        dy = p2.y - p1.y
        if abs(dx) > 1e-9:
            for x_edge in [self.x1, self.x2]:
                t = (x_edge - p1.x) / dx
                if 0 < t < 1 and self.y1 < p1.y + t * dy < self.y2: return
True
        if abs(dy) > 1e-9:
            for y_edge in [self.y1, self.y2]:
                t = (y_edge - p1.y) / dy
                if 0 < t < 1 and self.x1 < p1.x + t * dx < self.x2: return
True
        return False


def manhattan_distance(p1, p2):
    return abs(p1.x - p2.x) + abs(p1.y - p2.y)


def is_path_legal(p1, p2, doors):
```

```
    mid1 = Point(p2.x, p1.y);
    legal1 = True
    for door in doors:
        if door.intersect_segment(p1, mid1) or
door.intersect_segment(mid1, p2):
            legal1 = False;
            break
    len1 = manhattan_distance(p1, p2) if legal1 else float('inf')

    mid2 = Point(p1.x, p2.y);
    legal2 = True
    for door in doors:
        if door.intersect_segment(p1, mid2) or
door.intersect_segment(mid2, p2):
            legal2 = False;
            break
    len2 = manhattan_distance(p1, p2) if legal2 else float('inf')

    if legal1 or legal2:
        return True, min(len1, len2), [p1, mid1, p2] if len1 <= len2 else
[p1, mid2, p2]
    return False, float('inf'), []


def shortest_avoiding_path(p1, p2, doors):
    legal, dist, path = is_path_legal(p1, p2, doors)
    if legal: return dist, 'direct', path
```

```
    detour_points = []
    for door in doors:
        for corner in [Point(door.x1, door.y1), Point(door.x1, door.y2),
Point(door.x2, door.y1),
                       Point(door.x2, door.y2)]:
            if corner not in detour_points: detour_points.append(corner)

    best_dist = float('inf');
    best_path = []
    for corner in detour_points:
        l1_legal, l1_dist, l1_path = is_path_legal(p1, corner, doors)
        l2_legal, l2_dist, l2_path = is_path_legal(corner, p2, doors)
        if l1_legal and l2_legal and (l1_dist + l2_dist < best_dist):
            best_dist = l1_dist + l2_dist;
            best_path = l1_path[:-1] + l2_path

    return (best_dist, 'detour', best_path) if best_dist != float('inf')
else (float('inf'), 'none', [])



def build_graph(points, doors):
    n = len(points);
    edges = []
    for i in range(n):
        for j in range(i + 1, n):
            dist, _, path = shortest_avoiding_path(points[i], points[j],
doors)
            if dist != float('inf'): edges.append((i, j, dist, path))
```

```
    return edges



class DSU:
    def __init__(self, size):
        self.parent = list(range(size));
        self.rank = [0] * size

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]];
            x = self.parent[x]
        return x

    def union(self, x, y):
        x_root = self.find(x);
        y_root = self.find(y)
        if x_root == y_root: return False
        if self.rank[x_root] < self.rank[y_root]:
            self.parent[x_root] = y_root
        else:
            self.parent[y_root] = x_root
            if self.rank[x_root] == self.rank[y_root]: self.rank[x_root]
+= 1
        return True



def kruskal_mst(num_nodes, edges):
```

```
    edges.sort(key=lambda e: e[2])
    dsu = DSU(num_nodes);
    mst_edges = [];
    total_w = 0.0
    for u, v, w, path in edges:
        if dsu.union(u, v):
            mst_edges.append((u, v, w, path));
            total_w += w
            if len(mst_edges) == num_nodes - 1: break
    return (total_w, mst_edges) if len(mst_edges) == num_nodes - 1 else
(None, [])



# ==================== 第 2 部分：GUI 界面逻辑 ====================
class WiringApp:
    def __init__(self, root):
        self.root = root
        self.root.title("最小生成树应用：室内布线系统（前端界面版）")
        self.root.geometry("900x600")  # 设置窗口大小的宽 x 高

        # 左侧控制面板（给输入框留位置）
        control_frame = tk.Frame(root, width=250, padx=15, pady=15)
        control_frame.pack(side=tk.LEFT, fill=tk.Y)

        # 1. 插座输入区
        tk.Label(control_frame, text="💡 插座坐标 (每行一个，x y):",
font=("黑体", 11)).pack(anchor="w", pady=(0, 5))
        self.sockets_text = tk.Text(control_frame, height=8, width=25,
```

```
font=("Consolas", 10))
        self.sockets_text.pack(pady=5)
        # 预设几个坐标方便测试
        self.sockets_text.insert(tk.END, "0 0\n2 5\n8 0\n10 5")

        # 2. 门输入区
        tk.Label(control_frame, text="🚪 门坐标 (每行一个, x1 y1 x2 y2):",
font=("黑体", 11)).pack(anchor="w",

pady=(10, 5))
        self.doors_text = tk.Text(control_frame, height=5, width=25,
font=("Consolas", 10))
        self.doors_text.pack(pady=5)
        # 预设一个门挡在中间
        self.doors_text.insert(tk.END, "4 -1 6 3")

        # 3. 运行按钮
        self.calc_btn = tk.Button(control_frame, text="⚡ 开始计算并绘制路
径",
                                  command=self.calculate_and_draw,
                                  bg="#4CAF50", fg="white", font=("黑体",
12, "bold"))
        self.calc_btn.pack(pady=20, fill=tk.X)

        # 4. 结果文字展示区
        self.result_label = tk.Label(control_frame, text="等待计算...",
fg="#E91E63", justify=tk.LEFT,
                                     font=("黑体", 11))
```

```
        self.result_label.pack(anchor="w")


        # 右侧画板区（显示图像的内容）
        self.plot_frame = tk.Frame(root, bg="white", bd=2,
relief=tk.GROOVE)
        self.plot_frame.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True,
padx=10, pady=10)


        # 初始化 matplotlib 画布
        self.fig, self.ax = plt.subplots(figsize=(6, 5))
        self.canvas = FigureCanvasTkAgg(self.fig, master=self.plot_frame)
        self.canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)
        # 先空着画布提示一下
        self.ax.text(0.5, 0.5, "请在左侧输入数据后点击计算",
horizontalalignment='center', verticalalignment='center',
                     color='gray')
        self.ax.axis('off')


    # 当按下计算按钮时，会触发这个函数
    def calculate_and_draw(self):
        points = []
        doors = []


        # -- 尝试读取你在界面输入框里的文字 --
        try:
            # 读取插座数据
            socket_lines = self.sockets_text.get("1.0",
tk.END).strip().split('\n')
```

```
            for line in socket_lines:
                if line.strip():
                    x, y = map(float, line.split())
                    points.append(Point(x, y))

            # 读取门数据
            door_lines = self.doors_text.get("1.0",
tk.END).strip().split('\n')
            for line in door_lines:
                if line.strip():
                    x1, y1, x2, y2 = map(float, line.split())
                    doors.append(Rect(x1, y1, x2, y2))
        except Exception as e:
            messagebox.showerror("输入错误","数据格式不对！请确保输入数字且
用空格隔开。")
            return

        if len(points) < 2:
            messagebox.showwarning("提示", "你需要输入至少两个插座才能连线
哦！")
            return

        # -- 调用原有的算法计算连通和最小树 --
        edges = build_graph(points, doors)
        total_len, mst_edges = kruskal_mst(len(points), edges)

        if total_len is None:
            self.result_label.config(text="失败：被门彻底阻挡\n 无法全部连
```

```
通！")
            messagebox.showerror("计算失败", "所有插座无法通过可行路径连通（图不连通），请检查门是否太大阻挡了路径。")
            return

        # -- 更新左侧的计算结果文字 --
        self.result_label.config(text=f"✅ 布线成功！\n\n📌 最小生成树总长:\n{total_len:.2f}")

        # -- 将计算结果画到右侧面板上 --
        self.ax.clear()  # 先擦除旧图片
        self.ax.axis('on')  # 开启坐标轴

        # 1. 涂抹红色半透明矩形代表门
        for door in doors:
            rect = patches.Rectangle((door.x1, door.y1), door.x2 - door.x1, door.y2 - door.y1, color='red', alpha=0.3)
            self.ax.add_patch(rect)

        # 2. 点上蓝色插孔代表插座
        for i, p in enumerate(points):
            self.ax.scatter(p.x, p.y, c='blue', s=80, zorder=5)
            self.ax.text(p.x + 0.2, p.y + 0.2, f"插座{i + 1}", fontsize=10, fontweight='bold', color='darkblue')

        # 3. 按照算好的路径连上绿线
        for u, v, w, path in mst_edges:
            xs = [p.x for p in path]
```

```
            ys = [p.y for p in path]
            self.ax.plot(xs, ys, 'green', linewidth=2.5, alpha=0.8)

        # 4. 图表美化
        self.ax.grid(True, linestyle='--', alpha=0.6)
        self.ax.set_title("室内最小布线方案（曼哈顿距离）", fontsize=14,
fontweight='bold')
        self.ax.set_xlabel("X 向坐标")
        self.ax.set_ylabel("Y 向坐标")

        self.canvas.draw()  # 刷新画布，将图呈现出来


if __name__ == "__main__":
    # --- 解决 matplotlib 中文字体显示方格的问题 ---
    plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'Arial
Unicode MS']  # 适配不同系统的中文字体
    plt.rcParams['axes.unicode_minus'] = False  # 正常显示负号

    # --- 启动 Tkinter 应用 ---
    root = tk.Tk()
    app = WiringApp(root)
    root.mainloop()  # 让窗口持续显示
```